

Numerical Methods Lab

Lab 11 – 18 | Python Programs

Python Source Code

Lab-11: Newton's Forward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's forward difference. Data: $x=1, 1.2, 1.4, 1.6$ $y=0, 0.128, 0.544, 1.296$ at $x=1.1$

C Program:

```
import math
n = int(input("Enter number of data points: "))
x = []
y = []

print("Enter x and y values (x y):")
for i in range(n):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)

xp = float(input("Enter the value of x at which derivative is required: "))
h = x[1] - x[0]
diff = [y]
for i in range(1, n):
    row = []
    for j in range(n - i):
        row.append(diff[i-1][j+1] - diff[i-1][j])
    diff.append(row)
u = (xp - x[0]) / h
dy = (diff[1][0] + ((2*u - 1) / 2) * diff[2][0] + ((3*u*u - 6*u + 2) / 6) * diff[3][0]) / h
print("\nFirst derivative at x =", xp, "is", dy)
```

Lab-12: Newton's Backward Difference — First Derivative

Question: Write a Python and C program for calculating first derivative using Newton's backward difference. Data: $x=1, 1.2, 1.4, 1.6$ $y=0, 0.128, 0.544, 1.296$ at $x=1.1$

C Program:

```
n = int(input("Enter number of data points: "))
x = []
y = []

print("Enter x and y values (x y):")
for i in range(n):
    xi, yi = map(float, input().split())
    x.append(xi)
    y.append(yi)

xp = float(input("Enter the value of x at which derivative is required: "))
h = x[1] - x[0]
diff = [[0 for _ in range(n)] for _ in range(n)]

for i in range(n):
    diff[i][0] = y[i]
    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            diff[i][j] = diff[i][j - 1] - diff[i - 1][j - 1]
    u = (xp - x[n - 1]) / h
    dy = (diff[n - 1][1] + ((2 * u + 1) / 2)
          * diff[n - 1][2] + ((3 * u * u + 6 * u + 2) / 6)
          * diff[n - 1][3]) / h
    print("First derivative at x =", xp, "is", dy)
```

Lab-13: Simpson's 1/3 Rule

Question: Write a Python and C program for Composite Simpson's 1/3 Rule. $f(x) = x^2 + 2x + 1$, $x_0=0$, $x_n=4$, segments=4

C Program:

```
def f(x):
    return x**2 + 2*x + 1

def composite_simpsons_1_3_rule(x0, xn, k):
    if k % 2 != 0:
        print("Warning: Number of segments (k) must be even for Simpson's 1/3 rule.")
        return None

    h = (xn - x0) / k
    term = f(x0) + f(xn)

    for i in range(1, k, 2):
        term = term + 4 * f(x0 + i * h)

    for i in range(2, k - 1, 2):
        term = term + 2 * f(x0 + i * h)

    v = (h / 3) * term
    return v

print(f"Function: f(x) = x^2 + 2x + 1")
x0 = float(input("Enter lower limit (x0): "))
xn = float(input("Enter upper limit (xn): "))
k = int(input("Enter number of segments: "))

result = composite_simpsons_1_3_rule(x0, xn, k)

print("Approximate Integral =", result)
```

Lab-14: Simpson's 3/8 Rule

Question: Write a Python and C program for Simpson's 3/8 Rule. $f(x) = x^2 + 2x + 1$, $x_0=0$, $x_n=3$, segments=3

C Program:

```
def f(x):
    return x**2 + 2*x + 1

def simpsons_3_8_rule(x0, xn, k):
    if k % 3 != 0:
        print("Warning: For Simpson's 3/8 rule, the number of segments (k) should ideally be a multiple of 3.")

    h = (xn - x0) / k

    term = f(x0) + f(xn)

    for i in range(1, k):
        if i % 3 != 0:
            term = term + 3 * f(x0 + i * h)
        else:
            term = term + 2 * f(x0 + i * h)
    v = (3/8) * h * term
    return v

print(f"Function: f(x) = x^2 + 2x + 1")
x0 = float(input("Enter lower limit (x0): "))
xn = float(input("Enter upper limit (xn): "))
k = int(input("Enter number of segments: "))

integral_value = simpsons_3_8_rule(x0, xn, k)

print(f"Approximate integral using Simpson's 3/8 Rule: {integral_value}")
```

Lab-15: Romberg Integration

Question: Write a Python and C program for Romberg Integration. Integrand: $\sin(x)/x$, $x_0=0$, $x_n=1$, $p=4$ refinements, $q=3$ extrapolations

C Program:

```
import math

def integrand(x):
    if x == 0:
        return 1.0
    return math.sin(x) / x

def romberg_integration(f, x0, xn, p_max, q_max):
    h_initial = xn - x0

    T = [[0.0 for _ in range(p_max + 1)] for _ in range(p_max + 1)]
    T[0][0] = (h_initial / 2) * (f(x0) + f(xn))

    for i in range(1, p_max + 1):
        current_h_i = h_initial / (2**i)

        sum_f_midpoints = 0
        for k_sum in range(1, 2**(i - 1) + 1):
            x_val = x0 + (2 * k_sum - 1) * current_h_i
            sum_f_midpoints += f(x_val)

        T[i][0] = (T[i-1][0] / 2) + (current_h_i * sum_f_midpoints)

    for c in range(1, p_max + 1):
        for k in range(1, c + 1):
            if k <= q_max:
                factor_4k = 4**k
                T[c][k] = (factor_4k * T[c][k-1] - T[c-1][k-1]) / (factor_4k - 1)
            else:
                break

    display_table_rows = p_max + 1
    display_table_cols = q_max + 1
    truncated_T = [row[:display_table_cols] for row in T[:display_table_rows]]

    return T[p_max][q_max], truncated_T

a = float(input("Enter lower limit (x0): "))
b = float(input("Enter upper limit (xn): "))
num_trap_refinements = int(input("Enter number of trapezoidal refinements (p): "))
num_extrapolations = int(input("Enter number of extrapolations (q): "))

approx_integral, romberg_table = romberg_integration(integrand, a, b, num_trap_refinements,
num_extrapolations)

print(f"Target Romberg estimate T[{num_trap_refinements}][{num_extrapolations}]")
print(f"Approximate integral of sin(x)/x from {a} to {b}: {approx_integral:.10f}")

print("\nRomberg Table (T[i][j]):")
for row_idx, row in enumerate(romberg_table):
    row_str = " ".join([f"{val:.10f}" for val in row])
    print(f"T[{row_idx}]: {row_str}")
```

Lab-16: Gaussian Elimination

Question: Write a Python and C program for Gaussian Elimination. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Program:

```
def gaussian_elimination(A, B):
    n = len(B)

    for i in range(n):
        if abs(A[i][i]) < 1e-12:
            raise ValueError(f"Zero pivot found at row {i}")

        for j in range(i + 1, n):
            ratio = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= ratio * A[i][k]
            B[j] -= ratio * B[i]

    x = [0 for _ in range(n)]
    for i in range(n - 1, -1, -1):
        sum_ax = 0
        for j in range(i + 1, n):
            sum_ax += A[i][j] * x[j]
        x[i] = (B[i] - sum_ax) / A[i][i]

    return x

n = int(input("Enter order of square matrix (n): "))
A = []
B = []

print("Enter coefficients of the matrix:")
for i in range(n):
    row = []
    for j in range(n):
        val = float(input(f"A[{i+1}][{j+1}]: "))
        row.append(val)
    A.append(row)

print("\nEnter constants of vector b:")
for i in range(n):
    val = float(input(f"b[{i+1}]: "))
    B.append(val)

try:
    solution = gaussian_elimination(A, B)
    print("\nSolution of the system:")
    print(f"x = {solution[0]:.2f}")
    print(f"y = {solution[1]:.2f}")
    print(f"z = {solution[2]:.2f}")
except ValueError as e:
    print("Error:", e)
```

Lab-17: Gaussian Elimination with Partial Pivoting

Question: Write a Python and C program for Gaussian Elimination with Partial Pivoting. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Program:

```
def gaussian_elimination_pivoting(A, B):
    n = len(B)

    for i in range(n):
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k

        A[i], A[max_row] = A[max_row], A[i]
        B[i], B[max_row] = B[max_row], B[i]

        if abs(A[i][i]) < 1e-12:
            raise ValueError("The system is singular (no unique solution).")

        for j in range(i + 1, n):
            ratio = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= ratio * A[i][k]
            B[j] -= ratio * B[i]

    x = [0 for _ in range(n)]
    for i in range(n - 1, -1, -1):
        sum_ax = sum(A[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (B[i] - sum_ax) / A[i][i]

    return x

n = int(input("Enter order of square matrix (n): "))
A = []
B = []

print("Enter coefficients of 3x3 matrix:")
for i in range(n):
    row = []
    for j in range(n):
        val = float(input(f"A[{i+1}][{j+1}] = "))
        row.append(val)
    A.append(row)

print("\nEnter constants vector:")
for i in range(n):
    val = float(input(f"B[{i+1}] = "))
    B.append(val)

try:
    solution = gaussian_elimination_pivoting(A, B)
    print("\nSolution of the system:")
    print(f"x = {round(solution[0])}")
    print(f"y = {round(solution[1])}")
    print(f"z = {round(solution[2])}")
except ValueError as e:
```



```
print("Error:", e)
```

Lab-18: Gauss-Jordan Elimination

Question: Write a Python and C program for Gauss-Jordan Elimination. System: $x+y+z=6$, $2x+3y+z=14$, $x+2y+3z=14$

C Program:

```
def gauss_jordan(A, B):
    n = len(B)

    for i in range(n):
        max_row = i
        for k in range(i + 1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k

        A[i], A[max_row] = A[max_row], A[i]
        B[i], B[max_row] = B[max_row], B[i]

        if abs(A[i][i]) < 1e-12:
            raise ValueError("The matrix is singular or nearly singular.")

        pivot = A[i][i]
        for k in range(i, n):
            A[i][k] /= pivot
            B[i] /= pivot

        for j in range(n):
            if i != j:
                ratio = A[j][i]
                for k in range(i, n):
                    A[j][k] -= ratio * A[i][k]
                B[j] -= ratio * B[i]

    return B

n = int(input("Enter order of square matrix (n): "))
A = []
B = []
print("Enter coefficients of 3x3 matrix:")
for i in range(n):
    row = []
    for j in range(n):
        val = float(input(f"A[{i+1}][{j+1}] = "))
        row.append(val)
    A.append(row)

print("\nEnter constants vector:")
for i in range(n):
    val = float(input(f"B[{i+1}] = "))
    B.append(val)

try:
    solution = gauss_jordan(A, B)
    print("\nGauss-Jordan Solution:")
    print(f"x = {round(solution[0])}")
    print(f"y = {round(solution[1])}")
    print(f"z = {round(solution[2])}")
```

```
except ValueError as e:  
    print("Error:", e)
```